

Predicting Earnings Surprises from Analyst Revision Sequences

Rayyan Hussain
rhussai9@depaul.edu

Akshaj Sriachutananda
asriachu@depaul.edu

Damian Moreno
dmoren37@depaul.edu

June 5, 2026

Abstract

When a company reports quarterly earnings, the stock reaction depends less on the raw number and more on whether it beat or missed what analysts expected. The usual benchmark is the analyst consensus, a simple average that ignores which analyst made each forecast. We test whether a transformer that learns a separate embedding for each analyst can pull signal out of the individual estimates that the consensus average throws away. We built the model on several years of S&P 100 earnings events from the Bloomberg Terminal and compared it against naive baselines, a linear regression, and an LSTM. The honest result is a weak one. The transformer cannot predict the size of a surprise any better than just guessing the average ($R^2 \approx 0$), but it does produce a faint ranking signal (information coefficient of 0.11) that beats the linear baseline by about five times. Turning the analyst identity embeddings on or off makes no significant difference, which is a direct negative answer to our central question of whether per-analyst embeddings can pull signal out of the individual estimates that the consensus hides. The transformer, the LSTM, and the linear model all land in roughly the same place. We argue the limiting factor is the data we could actually extract, not the model: about four analysts per quarter, mostly mid-tier firms, captured as snapshots rather than the full revision history we proposed. The approach is therefore likely to do better with richer data, and future work would fold in that data along with hyperparameter tuning aimed at generalizing on the consensus.

1 Introduction

Every quarter, public companies report how much money they made. The stock price then moves, but not because of the absolute number. It moves on the gap between the actual result and what investors were expecting. The standard measure of that expectation is the analyst consensus, the average of forecasts published by analysts at firms like Goldman Sachs and Morgan Stanley. The earnings surprise is the difference between the reported number and this consensus.

Because the consensus is a plain average, it discards the underlying detail, which can matter for understanding how a stock moves at a given moment. It treats a stale forecast the same as a fresh one, treats a sloppy analyst the same as a sharp one, and smooths over patterns like a sudden burst of upward revisions in a single week. Prior work has shown the consensus is biased in predictable ways, which hints that there is real structure in the underlying estimates that averaging hides.

Our question is narrow and concrete. If we give a model the individual analyst estimates instead of their average, and we let it learn who each analyst is, can it do better than the consensus and better than simpler models? Our contribution is not a winning model. It is a careful, honest test of a specific hypothesis, with three baselines, an LSTM comparison, a direct embedding ablation, an attention analysis, and proper significance testing. The main finding is a clean negative one: at the data depth we could obtain, analyst identity does not help, and a faint ranking signal is the only thing that survives. We think that result is worth reporting precisely because the point of the project was to test the idea, not to assume it would work.

2 Related Work

Three lines of work set up our question. First, the analyst forecasting literature shows the consensus hides systematic structure. Clement and Tse [2] find that many analysts herd, clustering their estimates near the existing consensus instead of fully using their own information, and that bolder forecasts tend to be more accurate. Cowen et al. [3] trace analyst optimism mainly to sales and trading incentives rather than banking relationships, which again points to predictable bias in published forecasts. Brown and Mohammad [1] show that analyst identity carries information

beyond just how recent a forecast is; this does not necessarily help predict the next earnings surprise, but certain analyst characteristics can still be useful for other purposes.

Second, the work on star analysts shows that some forecasters stay more accurate over time [5], which is the clearest argument that analyst identity should carry signal that the average washes out. This is exactly what our embedding ablation is built to test.

Third, deep learning has been applied widely to financial time series, but work aimed specifically at the analyst estimate sequence is thin. We build on these threads by using a transformer [6] to learn both who to trust and how to weight the estimates, and we report a direct test of whether the identity part helps at all.

3 Preliminary and Background

This section defines the terms a reader outside finance or machine learning would need to follow the rest of the paper.

Earnings, EPS, and the consensus. Every quarter a public company reports its profit. A common way to express that profit is earnings per share, or EPS, which is the total profit divided by the number of shares, so it is the profit attributable to a single share of stock. Before the company reports, professional stock analysts at banks and research firms each publish their own guess for that EPS figure. The analyst consensus is simply the average of all those guesses. It is the market’s standard expectation for what the company will report.

The earnings surprise. The earnings surprise is the gap between what the company actually reported and what the consensus expected, written as a percentage of the consensus. A positive surprise means the company beat expectations, a negative surprise means it missed. We express it as a percent so that companies of different sizes can be compared on the same scale. Predicting this surprise is the central task of the project. The whole idea we are testing is that the individual analyst guesses, and the question of which analyst made each one, contain useful information that gets thrown away when you average them all into a single consensus number.

Transformers, attention, and embeddings. A transformer is a type of neural network, the same family of model that powers tools like ChatGPT. Its key feature is attention, a mechanism that lets the model look at a whole group of inputs at once and decide, for each input, how much weight to give every other input. In our case the inputs are the analyst guesses for one quarter, and attention lets the model decide which analysts to lean on. An embedding is a learned list of numbers that the model attaches to each analyst, the same way a language model attaches a vector of numbers to each word. The embedding is meant to capture an analyst’s track record and tendencies, so the model can in principle learn that some analysts are sharper or more biased than others.

How we judge the models. Beyond plain prediction error, we use a few standard measures. RMSE (root mean squared error) and MAE (mean absolute error) both measure how far predictions land from the truth on average, with RMSE punishing big misses more. R-squared measures how much of the variation in surprises the model actually explains, where zero means it explains nothing more than guessing the average. The information coefficient (IC) is the correlation between the predicted surprises and the real ones; it measures whether the model ranks events in the right order, even if it gets the exact sizes wrong. Directional accuracy is how often the model gets the sign right (beat versus miss), but it is misleading when almost all surprises are positive, so we also report balanced accuracy, which scores the model separately on beats and on misses and then averages the two, exposing a model that just always guesses “beat.” The Diebold-Mariano test is a statistical check of whether one model’s errors are significantly smaller than another’s, rather than smaller by luck. Finally, because we are forecasting the future, we train on the earliest events and test on the latest ones, never the reverse, so the model can never accidentally peek at information that would not have existed at prediction time.

4 Methodology

Input representation. Each earnings event is one training example. The input is the set of analyst estimates for that quarter. Each estimate becomes a vector that combines a learned 16-dimensional embedding for the analyst with four continuous features: the estimate value, its distance from the quarter mean, that distance normalized by the spread of estimates, and the prior quarter’s reported figure. Continuous features are standardized using statistics fit on the training set only.

Architecture. The forward pass is short and is the same for both prediction heads:

1. For each analyst, concatenate the continuous features with that analyst’s embedding.
2. Project each combined vector into a 64-dimensional space and prepend a learnable summary (CLS) token.
3. Pass the set through a two-layer transformer encoder with four attention heads, using a padding mask so empty slots are ignored.
4. Read off the summary token and feed it to two small heads, one predicting the percent surprise and one predicting the post-earnings return.

We removed positional encoding, because the estimates for a quarter arrive as an unordered snapshot rather than a time-ordered sequence, so position carries no meaning (see Section 5). The model has 76,690 parameters. We train it with the Huber loss, a common choice that behaves like ordinary squared error for small misses but switches to a gentler absolute-error penalty for large ones, so a few wild outliers cannot dominate training. The loss is summed over both prediction heads, with the return head down-weighted to half, which gives the model a light secondary signal without letting it distract from the main surprise target. Optimization uses AdamW, a standard variant of gradient descent, with a one-cycle learning rate schedule, gradient clipping for stability, and early stopping once validation error stops improving.

Baselines. We compare against four reference points of increasing complexity. Predict-zero assumes no surprise, which is the same as taking the consensus at face value. Predict-mean guesses the average training surprise for every event. A ridge regression, which is ordinary linear regression with a penalty that discourages extreme coefficients, learns from per-quarter summary statistics of the estimates, and it is the most important baseline because it asks whether a transformer is needed at all. An LSTM [4], an older type of neural network that reads inputs one at a time and was the standard before transformers, uses the same inputs and isolates whether attention specifically helps over that earlier approach.

5 Numerical Experiments

Data gathering and processing. We pulled quarterly earnings data for S&P 100 companies from the Bloomberg Terminal in DePaul’s John L. Keeley Jr. Finance Lab, exporting through the Excel Add-in into per-stock CSV files: one earnings-history file and one per-analyst estimate file per ticker. Processing involved cleaning Bloomberg’s formatting (dropping summary rows, parsing percent strings, normalizing period labels like “Q1 25” and “2025 Q1” to a common form), then merging each stock’s earnings history with its analyst estimates on ticker and quarter. We engineered the per-estimate features described above and clipped extreme surprises to $\pm 50\%$ so that a few outliers, such as the COVID quarter, would not dominate the loss. After cleaning we had 92 stocks, 1,766 quarter-announcements, and 6,936 analyst-estimate rows, which is roughly four analyst estimates per quarter. The usable history averages about 19 quarters per stock; the raw earnings record for some names runs back to around 2015, though the per-analyst estimate coverage that actually gates our sample is shorter. The Bloomberg license does not allow redistribution, so we describe the data here but do not attach it.

Two facts about this dataset matter. First, it is heavily skewed: 87.6% of surprises are positive, because firms tend to beat a consensus that has been guided down. Second, and more important, we did not get what the proposal asked for. We proposed a time-ordered sequence of 30 to 80 revisions per event including the bulge-bracket analysts (the analysts at the largest and most prestigious Wall Street banks, such as Goldman Sachs, Morgan Stanley, and JPMorgan), but that depth was not extractable through our subscription. What we could pull was a snapshot of the standing estimates per quarter, about four per event, mostly from mid-tier and boutique firms with no bulge-bracket coverage. This is why the model is cross-sectional rather than a revision-sequence model, and why positional encoding was dropped. We split strictly by date into 1,219 training, 261 validation, and 267 test events.

Results. Table 1 collects the test-set results. The clearest way to read them is to separate error from ranking, because the model behaves very differently on the two.

Magnitude. The transformer’s test RMSE is 12.44, and just predicting the mean gives 12.43. Those are the same number, and the R^2 is -0.002 , so the model explains essentially none of the variance in surprise size. A Diebold-Mariano test against predict-mean gives $p = 0.80$, a clear tie. The model does beat predict-zero ($p < 0.0001$), but that is a low bar, since predict-zero ignores the obvious positive skew.

Ranking. Where the model shows life is in ranking. Its Pearson information coefficient is 0.111 ($p = 0.07$, marginal, with a 95% confidence interval of roughly $[-0.01, 0.23]$), and the rank-based Spearman version is lower at

Model	RMSE	MAE	DirAcc	BalAcc	IC
Transformer (full)	12.44	7.54	0.876	0.50	0.111
LSTM	12.44	7.72	0.876	0.50	0.044
Ridge (summary stats)	13.60	8.55	0.873	0.498	0.019
Predict-mean	12.43	7.74	0.876	0.50	–
Predict-zero / consensus	14.10	9.16	0.00	0.00	–

Table 1: Test-set results on 267 earnings announcements. RMSE and MAE are in percentage points of surprise. DirAcc is raw directional accuracy, BalAcc is balanced accuracy across up and down surprises, and IC is the Pearson correlation between predicted and actual surprise. IC is left blank for the two constant-forecast baselines (predict-mean and predict-zero), because correlation is undefined when the prediction has no variance.

0.084 but still positive. The ridge baseline manages only 0.019 ($p = 0.75$, indistinguishable from noise), so on the Pearson measure the transformer ranks about five times better than the linear model, and it also carries lower error against that same ridge (RMSE 12.44 versus 13.60). The signal is faint and sits just above the usual significance threshold, and because it comes from a single training run we treat it as suggestive rather than established; confirming it would mean repeating the training across several random seeds.

Directional accuracy is a mirage. Directional accuracy is 0.876, which looks like a strong result. However, since 87.6% of surprises are positive, this simply reflects that the model predicts a positive surprise in essentially every instance. In context the result is weak and provides no true insight, which the balanced accuracy of 0.50, a coin flip, confirms. We report both side by side on purpose, because directional accuracy alone would badly overstate what the model does.

Analyst identity has no significance. This is the direct test of our hypothesis. We trained the model twice, once with the analyst embeddings and once with them removed, holding everything else fixed. Test errors are essentially identical (MSE approximately 154 across both tests), and a paired Wilcoxon test on the per-event squared errors finds no significant difference. At four analysts per quarter, knowing who made each estimate adds almost nothing. We believe that the biggest contributors to our reported results were the lack of data and the quality of analysts within our dataset. Our data was dominated by mid-tier and boutique analysts rather than the bulge-bracket names, whose reputations are strongest and whose identity would be the most likely to carry real signal. Because we never had access to that higher-quality coverage, the embeddings had little genuine identity information to learn from, so it is unsurprising that turning them off costs nothing.

Architecture and diagnostics. The LSTM lands at RMSE 12.44, the same spot as the transformer, so on error swapping attention for recurrence changes nothing. Its ranking IC of 0.044 sits between the linear baseline and the transformer, so the attention model keeps a small ranking edge, but both numbers are weak and neither is significant on its own. When three different model families produce very similar poor results, the problem is likely due to the data we used rather than the architecture. The attention weights are diffuse, with the top analyst in a typical event getting only about 0.15 of the weight and no single analyst dominating, which fits the weak embedding result. To further confirm our results, actual surprise correlates with actual post-earnings return at $r = 0.179$, the expected sign, while the model’s predicted surprise correlates with return at only $r = -0.087$, which follows from the model not predicting magnitude well.

6 Conclusion

We set out to test whether a transformer with learned analyst embeddings could beat the consensus and simpler models at predicting earnings surprises. On magnitude it cannot, tying the predict-mean baseline with an R^2 near zero. On ranking it shows a weak but real edge, with an information coefficient of 0.11 against the linear baseline’s 0.02. The analyst identity embeddings, which were the core of the idea, make no significant difference, and a transformer, an LSTM, and a ridge regression all converge to the same performance. These results were due to the lack and the quality of the data we used.

The limitations point directly at the cause. The hypothesis needs the rich part of the analyst record, the full sequence of who revised when including the bulge-bracket names that carry the most reputation, but we could only extract a shallow snapshot of about four mid-tier analysts per quarter with no ordering. With that input the consensus and the individual estimates are nearly the same object, so it is no surprise that identity buys little. The negative embedding result should be read as “identity does not help at this data depth,” not as proof that it never could. Smaller limitations include the heavy positive skew, which makes directional metrics misleading, the $\pm 50\%$

clipping, which slightly flattens error metrics, and the small 267-event test set, which leaves the marginal IC able to tip either way.

The most promising next step is therefore about data rather than architecture: obtaining fuller revision-level history with bulge-bracket coverage and per-revision timestamps, which would let the model train on more realistic data and likely give better results. With appropriate, more detailed data, the same transformer would finally get a fair test of the original hypothesis.

References

- [1] Lawrence D. Brown and Emad Mohammad. The predictive value of analyst characteristics. *Journal of Accounting, Auditing and Finance*, 18(4), 2003.
- [2] Michael B. Clement and Senyo Y. Tse. Financial analyst characteristics and herding behavior in forecasting. *The Journal of Finance*, 60(1):307–341, 2005.
- [3] Amanda Cowen, Boris Groyberg, and Paul Healy. Which types of analyst firms are more optimistic? *Journal of Accounting and Economics*, 41(1-2):119–146, 2006.
- [4] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [5] Scott E. Stickel. Reputation and performance among security analysts. *The Journal of Finance*, 47(5):1811–1836, 1992.
- [6] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

External Resources Statement

Per the course syllabus, full completion of this section is strictly required. It does not count toward the page limit. The team should bold only the options that accurately describe their own use before submitting.

- Beyond the course materials, what external resources did you use to complete this assignment?
 - Bloomberg Terminal documentation, used to determine which individual analyst estimate fields are accessible through the Excel Add-in and at what depth.
 - Google Scholar and journal websites, used to verify the bibliographic details and findings of the cited papers.
- If you used any AI tools, what model(s) and version number(s) did you use? **Claude Opus 4.8 (Anthropic)**. [Update if other tools or versions were also used.]
- If you used any AI tools, what interface(s) did you use? Highlight all that apply in bold.
 - **Web browser**
 - Mobile app
 - IDE/desktop app (e.g. VS Code + copilot, Cursor)
 - Command-line / terminal (e.g. Claude Code CLI)
 - Other (please specify):
 - I used no AI tools to complete this assignment.
- If you used any AI tools, how did you use them? Highlight all that apply in bold.
 - **Generate code: used verbatim in the final submission.**
 - **Generate code: modified or used as inspiration.**
 - **Generate text: multiple sentences used verbatim in the final submission.**
 - **Generate text: light editing or generating a few words or phrases.**
 - **Conceptual explanation: explain a specific concept or narrow question.**
 - **Conceptual explanation: high-level overview of a topic or general question.**
 - **Debugging: identify and explain errors in code.**
 - **Debugging: fix errors in code.**
 - Other (please specify):

- I used no AI tools to complete this assignment.
- If you used any AI tools, what context did you provide the tool(s)? Highlight all that apply in bold.
 - **Full assignment**
 - **A specific question or part of the assignment**
 - **A specific error message or bug**
 - **Only a written description of the problem**
 - Other (please specify):
 - I used no AI tools to complete this assignment.

A Project Background and Scope

The project grew out of a search for a finance problem that genuinely needed deep learning rather than a simpler model. We evaluated a range of candidates (deal premium prediction, bond rating changes, and others) against criteria like sample size, data accessibility through Bloomberg, and whether a sequence model would plausibly beat a tabular baseline. Earnings surprise prediction won because it has thousands of events, a well documented real-world signal, and a clean story for why analyst identity and revision order might matter. The intended universe was the S&P 100 over five years, which we estimated at roughly 100,000 analyst revision records across about 2,000 earnings events. The team of three split the work into data extraction, modeling, and experiments and writing, with weekly check-ins.

B Bloomberg Data Extraction

This was by far the hardest part of the project and is the direct cause of the data limitation discussed in the main text, so we document it in full.

What we were after. For each stock and quarter we wanted the individual broker-level EPS estimates, each tagged with the analyst and firm that produced it, ideally as a time-ordered history of revisions in the 90 days before each earnings announcement. The earnings history itself (reported EPS, consensus EPS, percent surprise, price reaction) was straightforward; the per-analyst detail was not.

What worked. The earnings history exported cleanly. Using the `ERN` function on a ticker, we pulled the full quarterly earnings record with consensus, actual, surprise, and price change, and saved it to CSV. The broker-level detail was visible on the terminal through the `EEB` screen (estimates consensus detail), which lists named analysts and their firms and rolls back to past quarters through a period selector; we verified that genuine historical data exists back several years. The `EEB` screen’s “Export to Excel” button does not write static numbers, it writes a live Bloomberg query formula, and that formula was the key to extraction. A single working formula for one stock and one quarter looked like `=BQL("AAPL US Equity", "F1396", "FPR=2023Q3", ..., "EST_SOURCE=DAD", ...)`, where `F1396` is the estimate field, `FPR` is the fiscal period (stepped to walk through time), and `EST_SOURCE` is a broker contributor code (for example `DAD` for D.A. Davidson, `GSR` for Goldman Sachs Research). An empty result for a given broker code simply means that broker does not cover that stock, which we treated as missing data rather than zero.

Where it broke down. Scaling this from one stock to the whole index was the wall. Three problems compounded. First, the terminal function codes (`EE`, `ERN`, `EEB`) are display functions and do not map onto queryable field names, so `F1396` is recognized by the worksheet `=BQL()` function but not by the more flexible `=BQL.Query()` and `BQL Builder` routes, which returned “unable to determine a data providing service” errors that point to a subscription-tier restriction on our library terminal. Second, when we swapped a single ticker for a multi-stock argument, the output collapsed to one value per stock instead of spilling one row per analyst, because the broker-detail structure was not preserved across the universe. Third, a shared monthly download cap on the library terminal made any brute-force loop over 100 stocks and 20 quarters risky to attempt. We confirmed that the two-quarter display limit on the `EEB` screen is only a screen limit and not a data limit, and that the bulk recommendations field `BEST_ANALYST_RECS_BULK` returns analyst ratings rather than EPS revision history, so it could not substitute. We also reached out through the terminal’s help desk for the correct broker-dimension query syntax.

The practical compromise. Faced with these limits and a fixed deadline, we fell back to what we could reliably extract: a per-quarter snapshot of the standing analyst estimates for each stock, harvested one stock at a time, rather than the intra-quarter revision sequence we had proposed. This is why the final dataset has about four analysts per quarter, is dominated by mid-tier and boutique firms with no bulge-bracket coverage, and contains no ordering information. Every modeling choice downstream, including the removal of positional encoding and the cross-sectional rather than sequential framing, follows from this compromise. The hardware was a standard library workstation running the Bloomberg Excel Add-in version 1.4.4.

C Data Processing and Feature Engineering

Each stock produced two CSV files, an earnings-history file and a per-analyst detail file, which we matched on ticker and quarter. Cleaning handled Bloomberg’s export quirks: a trailing “Average of Absolute Values” summary row was dropped, percent strings and non-meaningful markers such as “N.M.” were parsed into numbers or missing values, and quarter labels were normalized to a single form because the two files labeled periods differently (for example “Q1 25” versus “2025 Q1”). For each estimate we built four continuous features: the raw estimate value, its difference from the quarter’s mean estimate, that difference divided by the spread of estimates for the quarter (so it reads as a standardized position within the analyst group), and the prior quarter’s reported figure as a simple memory term. Continuous features were standardized using statistics computed on the training set only, so no test-set information leaked into the scaling. Surprises were clipped to plus or minus 50 percent and price changes to plus or minus 30 percent so that a handful of extreme quarters, such as the COVID period, could not dominate training.

D Model Architecture and Training Details

The transformer has 76,690 parameters: a 16-dimensional analyst embedding table, a linear projection from the 20-dimensional combined input (four continuous features plus the 16-dimensional embedding) to a 64-dimensional model space, two encoder layers with four attention heads and a 128-dimensional feedforward block, and two small prediction heads. A learnable summary token is prepended to the set of analyst vectors, and the model reads its final state as the pooled representation that feeds both heads. Positional encoding was removed because the snapshot has no meaningful order. Training used AdamW (learning rate 10^{-4} , weight decay 10^{-2}), a one-cycle learning rate schedule over 50 epochs, gradient norm clipping at 1.0, the Huber loss with its threshold set to 5, and a secondary return-head loss weighted at half the surprise loss. The analyst vocabulary was built from the training set only, with a single shared index reserved for analysts unseen in training. We trained on a Google Colab A100 GPU in PyTorch; a full run takes a few minutes. The LSTM baseline reuses the same inputs, loss, and training loop, swapping the encoder for a two-layer bidirectional recurrent network of comparable width.

E Evaluation Methodology

We report results on a strictly time-ordered test set so that the model is always judged on events later than anything it trained on. Beyond RMSE and MAE we computed the information coefficient (the correlation between predicted and actual surprise) for the transformer and the linear baseline, balanced accuracy alongside raw directional accuracy to expose the positive-skew problem, and a Diebold-Mariano test on the per-event squared errors to check whether differences between models were real or chance. The Diebold-Mariano test was run for the transformer against predict-zero, against predict-mean, and against the ridge baseline reported in Table 1. The embedding ablation trained the model with and without the analyst embeddings under a fixed random seed and compared per-event squared errors with a paired Wilcoxon signed-rank test, which is more appropriate than a t-test here because the surprise distribution is heavy-tailed. The attention analysis read the weights the summary token places on each analyst to see whether the model concentrated on a few names. To ensure consistency on the secondary head, we confirmed that actual surprises correlate positively with actual post-earnings returns, the direction finance theory predicts. The notebook also saves three diagnostic plots, the predicted-versus-actual scatter, the error distribution, and the surprise distribution, which together show predictions collapsing toward the positive mean rather than tracking the tails.